# CNENGR202 Instrumentation and Control

## Part 3 L2 – Computer Architecture

Prof. Zhang Liwei 张立伟 教授

Email: lwzhang@bjtu.edu.cn
Working Room: SX702

School of Electronics and Information Engineering
Beijing Jiaotong University

# Recap

- Embedded systems all around us!
- Microcontrollers are used to control engineering systems
- Think of microcontrollers as tiny computers where everything happens inside the chip
- Would like to control motors, leds, etc…
- Would like to read sensors and switches etc.

# Outline

- Computer architectures
  - Von Neumann architecture
  - Harvard architecture
  - Instruction set architecture
  - Memory hierarchy
- Atmega 328p

# Von Neumann Architecture

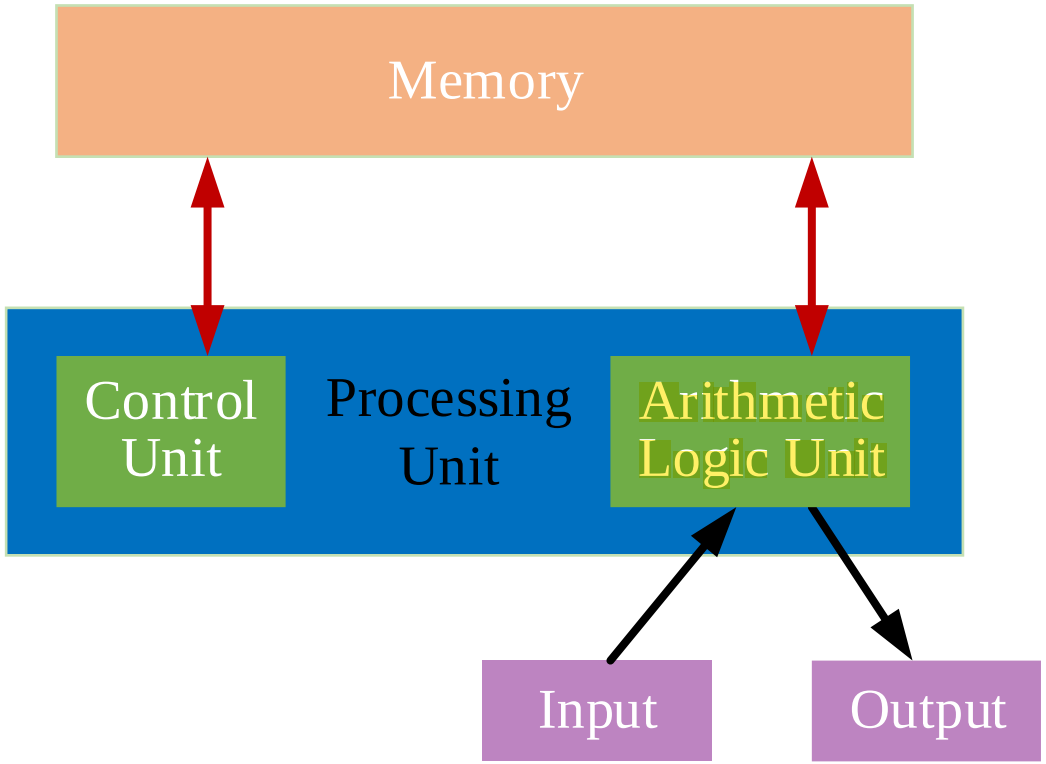
*LDI r16, 162*

What is data? What is an instruction?

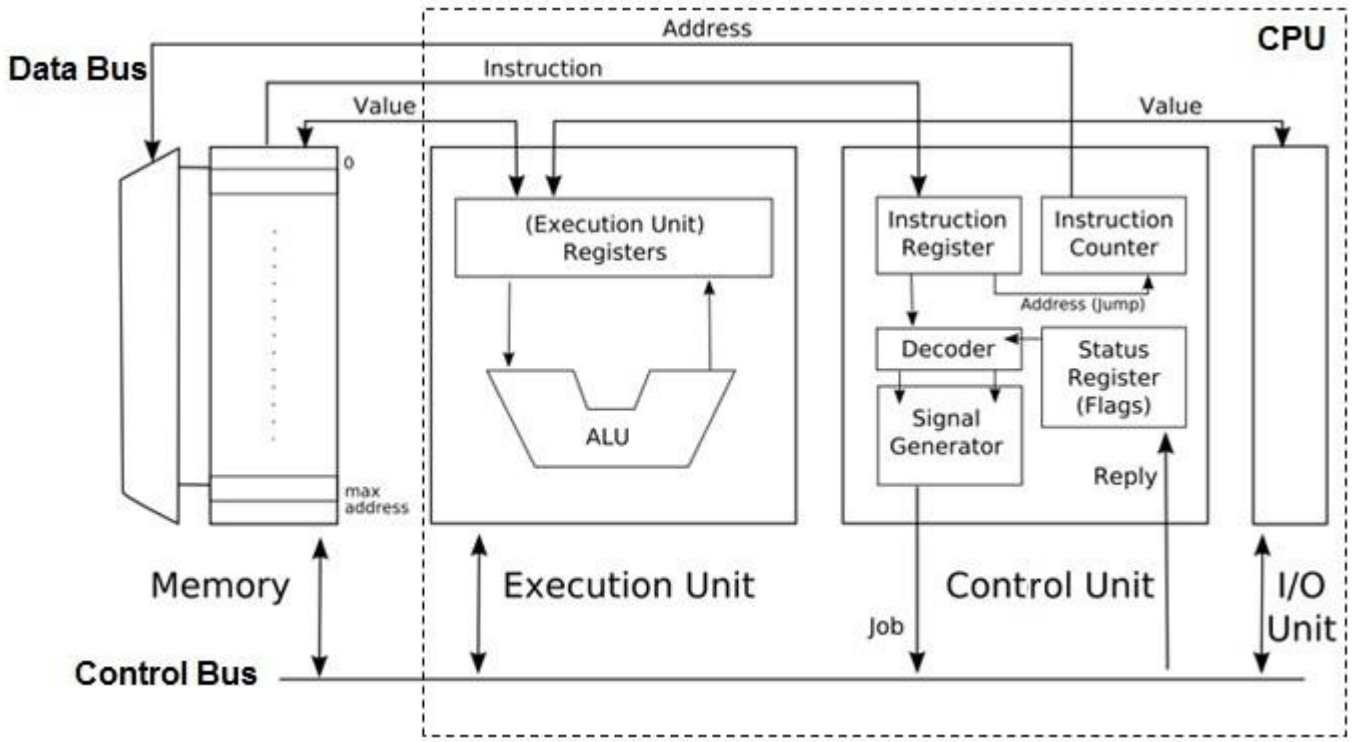
# Von Neumann Architecture

Important to note at this stage

- Both data AND instructions come from a single memory store, accessed through the control bus and sent through a data bus.
- There is NO WAY of distinguishing between data and program instructions. However, a compiler will usually separate them.



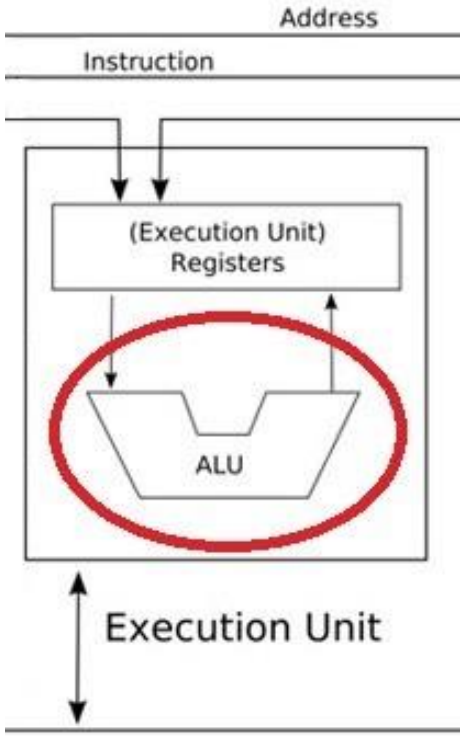
# Von Neumann Architecture




A common example of von Neumann architecture is the modern PC.

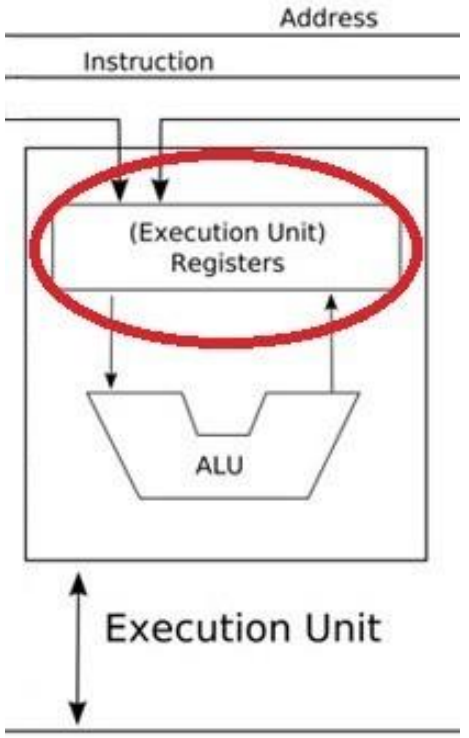
# Arithmetic/Logic Unit (ALU)

- An ALU is a physical digital circuit which performs specific arithmetic or logic operations on a register or set of registers.
- The task which the ALU performs is dictated by the ***instruction set*** of the CPU.
- An ALU must process numbers using the same format as the rest of the digital circuit. The format of modern processors is almost always the 'two's complement' binary number representation.
- A floating - point unit (*FPU*) also performs arithmetic operations, but for numbers in floating - point representation, much more complicated than the two's complement representation used in a typical ALU.



# Accumulator and General Purpose Registers

- In CPUs the accumulator is a special register which adds the result from the ALU automatically to the value already stored in it.
- Originally it was used as a way of storing intermediate arithmetic results without returning them to main memory (which slows things down).
- In modern CPUs general purpose registers are used for the same task – the instruction set tells the ALU which registers to use. The accumulator is then used mainly to set flags used for conditional jumping or branching in the status register.



# Status Register - SREG

- The status register contains information about the state of the processor.
- In a traditional processor design the status register includes at least three 'flags': Zero (Z), Carry (C), and Overflow (V), which are set or cleared automatically as side effects of arithmetic operations and may then be tested via conditional branch or jump instructions.
- A status register may often have other, more specialized flags, interrupt enable bits, and similar types of information.
- SREG Status Register

C: Carry Flag  Z: Zero Flag  N: Negative Flag  V: Two's Complement Overflow Flag

S: Sign Flag  H: Half Carry Flag  T: Transfer Bit  I: Global Interrupt Enable Bit

# Registers and Operands

Rd: Destination (and source) register in the Register File

Rr: Source register in the Register File

R: Result after instruction is executed

K: Constant data

k: Constant address

b: Bit position (0..7) in the Register File or I/O Register

s: Bit position (0..7) in the Status Register

X,Y,Z: Indirect Address Register
(X=R27:R26, Y=R29:R28, and Z=R31:R30
or X=RAMPX:R27:R26, Y=RAMPY:R29:R28,
and Z=RAMPZ:R31:R30
if the memory is larger than 64 KB)

A: I/O memory address

q: Displacement for direct addressing

UU: Unsigned × Unsigned operands

SS: Signed × Signed operands

SU: Signed × Unsigned operands

# Memory Space Identifiers

DS( ): Represents a pointer to address in data space

PS( ): Represents a pointer to address in program space

I/O(A) :I/O space address A

I/O(A,b): Bit position b of byte in I/O space address A

Rd(n): Bit n in register Rd

# Operator

$\times$ Arithmetic multiplication

$+$ Arithmetic addition

$-$ Arithmetic subtraction

$\wedge$ Logical AND

$\vee$ Logical OR

$\oplus$ Logical XOR

$>>$ Shift right

$<<$ Shift left

$==$ Comparison

$\leftarrow$ Assignment

$\leftrightarrow$ Swap

# Stack

STACK: Stack for return address and pushed registers

SP: The Stack Pointer

# Flags

⇔ Flag affected by instruction

0 Flag cleared by instruction

1 Flag set by instruction

- Flag not affected by instruction

# Instruction Register

- In simple CPUs the instruction register is used to hold the current instruction whilst it is decoded and executed.
- Decoding the instruction involves
  - getting the instruction
  - finding the operands in the memory
  - retrieving the operands
  - allocating processor resources for execution (in superscalar processors), etc.
- In more complex CPUs the instruction register is a 'pipeline' of instructions where each stage of the pipeline does part of the decoding, preparation or execution in a parallel process.

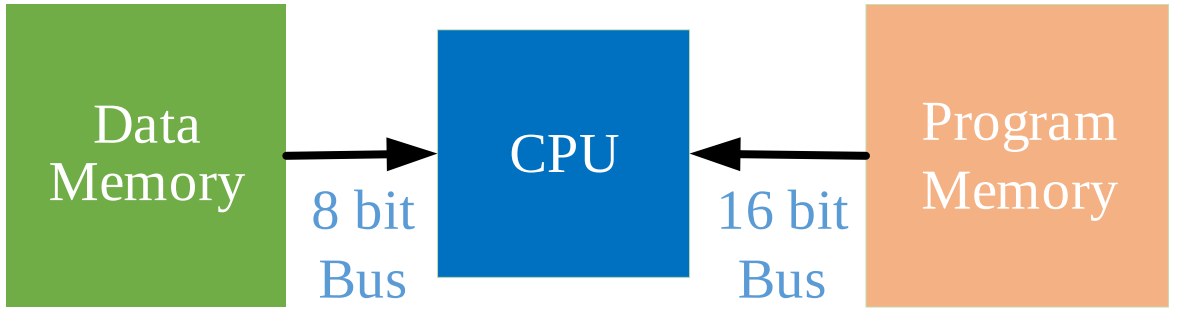
# Program Counter - PC

- The program counter (PC), also called the *instruction pointer*, the *instruction address register* ,the *instruction counter*, or just *part of the instruction sequencer*, is a CPU register which indicates where a computer is in its program sequence.
- The program counter holds the memory address of the next instruction to be executed. Once this has been read the program counter will automatically increment by one and point to the next instruction.
- Special instructions can rewrite the value of the program counter, so that code branching and looping can be achieved.

# Harvard Architecture

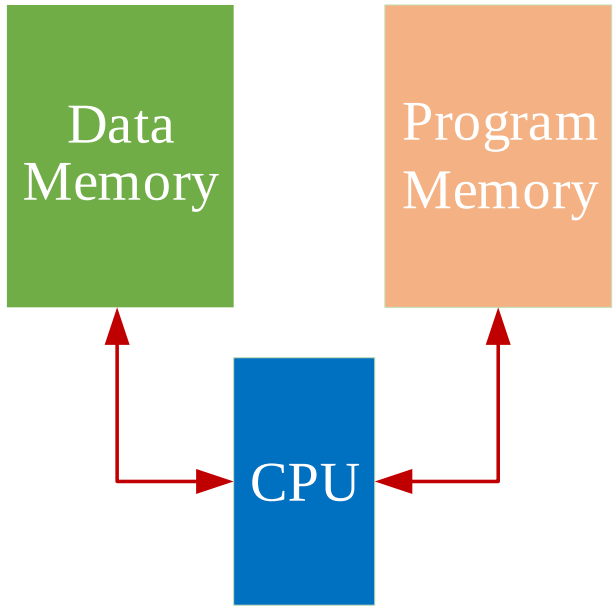
The important thing to note:

The data memory and program memory are separate, they are ***TWO DIFFERENT*** storages. The CPU addresses each store separately, therefore it will always be able to distinguish data memory from program memory .



# Harvard Architecture - I

- There is no need to make the two memories have the same characteristics.
  - Each one can use different lengths of words (i.e. numbers of bits representing instructions and data)
  - Different access timings
  - Be implemented using different technologies and memory address structures



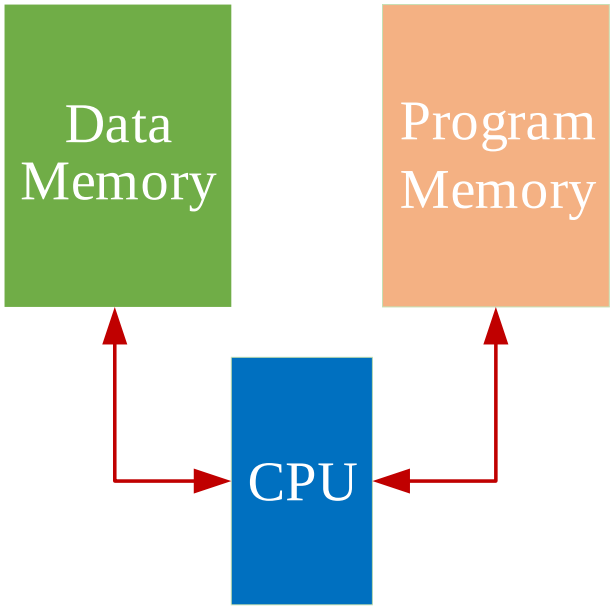
# Harvard Architecture - II

- Access two memory systems simultaneously, it can read both the program instruction and the data that it requires at the same time, using CPU time more efficiently.

- In contrast, the von Neumann processor has to read an instruction first, then read the data that the instruction requires.

- In addition to employing completely separate memory systems to store instructions and data, some implementations, such as microcontrollers, use different types of memory for the different applications

  (e.g. flash memory for the program instructions and, typically, read/write memory for the data).

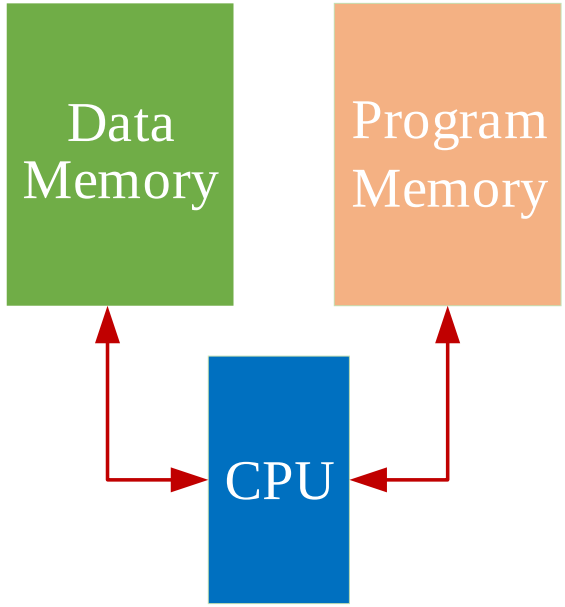


# Harvard Architecture - III

- However, if a number of programs are stored on the computer, the architecture must provide a way to distinguish between them and load the correct instructions into the instruction memory of the CPU.

- Also, having separate address spaces causes difficulties when programming with high - level languages such as C, which were originally developed for von Neumann architecture machines and don't support directly the use of different address spaces (e.g. tables of read - only data might be in a different address space from normal writable data, so need to be read using different instructions).

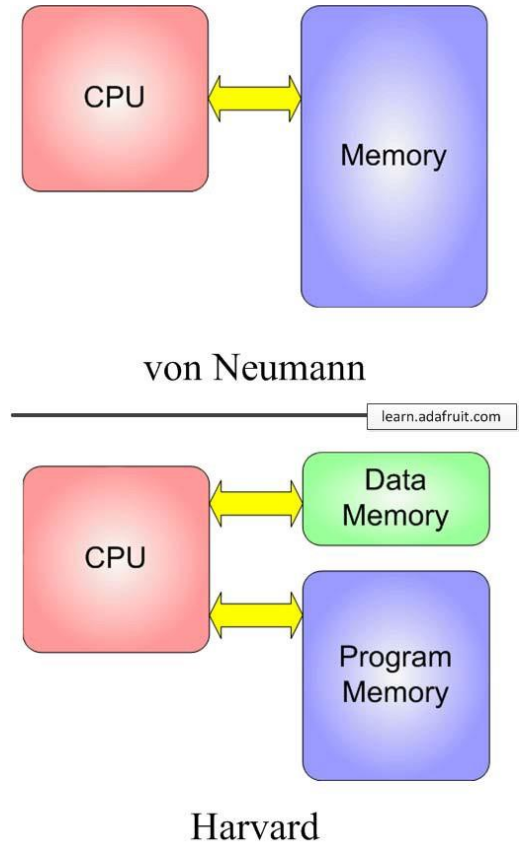
- Handled by the compiler



# Quick Summary

***To sum up then***

- A computer with von Neumann architecture (*and no cache*), stores *PROGRAM* memory and *DATA* memory in **THE SAME** location, so the CPU can either read an instruction *OR* read/write data from/to the memory.
- A computer with Harvard architecture stores *PROGRAM* memory and *DATA* memory in **SEPARATE** locations, so the CPU can read an instruction and perform a data memory access at the same time, even without a cache memory.



# Instruction Set Architecture

- ***Instruction set architecture (ISA)*** describes the processor in terms of what the assembly language programmer sees, the instructions and registers.
- Each processor has its own unique *instruction set*, a set of machine code instructions specifically designed to make best use of that processor.
- The actual number of instructions provided ranges from a few dozen for a simple 8 - bit microprocessor to several hundred for some 32 - bit processors.
- The two main processor designs are:
  - ***RISC*** (Reduced Instruction Set Computer) which use sets of relatively small instructions
  - ***CISC*** (Complex Instruction Set Computer) such as machines based on the Intel 8086 and Motorola 68000 microprocessor families.

# RISC Characteristics

There is no universally accepted definition, but most of the following apply:

- Most instructions are executed in a single machine cycle.
- All computational instructions involve registers and memory accesses are made with only load and store instructions ie load/store, register - to - register operation
- All instructions are the same length (typically 32 bits) and have just a few ways to address memory.

# RISC Characteristics

- Pipelining: The instruction set design allows for the processing of several instructions at the same time.
- High–performance memory: RISC machines have at least 32 general– purpose registers and a large cache memory.

# Arithmetic and Logic Instructions

| Mnemonic | operands | Description | Operation | Flags | Clocks AVRe |
|---|---|---|---|---|---|
| ADD | Rd, Rr | Add without Carry | Rd ← Rd + Rr | Z,C,N,V,S,H | 1 |

# Change of Flow Instructions

| Mnemonic | operands | Description | Operation | Flags | Clocks AVRe |
|---|---|---|---|---|---|
| RJMP | k | Relative Jump | PC ← PC + k + 1 | None | 2 |

# Data Transfer Instructions

| Mnemonic | operands | Description | Operation | Flags | Clocks AVRe |
|---|---|---|---|---|---|
| MOV | Rd, Rr | Copy Register | Rd ← Rr | None | 1 |

# Bit and Bit-Test Instructions

| Mnemonic | operands | Description | Operation | Flags | Clocks AVRe |
|---|---|---|---|---|---|
| LSR | Rd | Logical Shift Left | C ← Rd(7) | Z,C,N,V,H | 1 |

# MCU Control Instructions

| Mnemonic | operands | Description | Operation | Flags | Clocks AVRe |
|---|---|---|---|---|---|
| NOP | | No Operation | | None | 1 |

# Instruction Sets and example- 1

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| ADD Rd, Rr | Rd ← Rd+ Rr | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| ADDC Rd, Rr | Rd ← Rd+ Rr+C | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| SUB Rd, Rr | Rd ← Rd- Rr | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| SUBI Rd, k | Rd ← Rd- k | 16⩽d⩽31 0⩽k⩽255 | PC ← PC+1 |
| SBC Rd, Rr | Rd ← Rd- Rr-C | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| SBCI Rd, k | Rd ← Rd- k-C | 16⩽d⩽31 0⩽k⩽255 | PC ← PC+1 |

R16 = 0x10 R17=0x21 C=1 PC=0x200

ADD R16, R17 R16 = 0x30 R17=0x21 C=0 PC=0x201

ADDC R16, R17 R16 = 0x32 R17=0x21 C=0 PC=0x201

# Instruction Sets and example- 2

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| ADIW R[d+1]:Rd, K | R[d+1]:Rd ← R[d+1]:Rd+K | d∈{24, 26, 28, 30}<br>K⩽63 | PC ← PC+1 |
| SBIW R[d+1]:Rd, k | R[d+1]:Rd ← R[d+1]:Rd-k | d∈{24, 26, 28, 30}<br>k⩽63 | PC ← PC+1 |

R25 = 11 R24=23 C=1 PC=0x200

ADIW R25:R24, 5 R25 = 11 R24=38 C=0 PC=0x201

# Instruction Sets and example - 3

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| AND Rd, Rr | Rd ← Rd ∧ Rr | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| ANDI Rd, K | Rd ← Rd ∧K | 16⩽d⩽31 0⩽K⩽255 | PC ← PC+1 |

R2 = 0x15 R3=0x34 PC=0x200

AND R2, R3 R2 = 0x14 R3=0x34 PC=0x201

ANDI R3, 15 R2 = 0x14 R3=0x4 PC=0x201

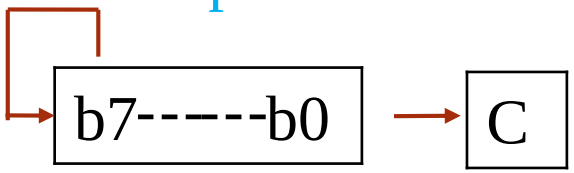
# Instruction Sets and example - 4

Syntax

ASR Rd

Arithmetic Shift Right

Operation




Operands

0⩽d⩽31

Program Counter

PC ← PC+1

R16 = 0x15 C=0 PC=0x200

ASR R16 R16 = 0xA C=1 PC=0x201

# Instruction Sets and example - 5

| Syntax | Operation | Operands |
|---|---|---|
| BRCC k | If C==0 then PC ← PC+k+1 else PC ← PC+1 | -64⩽k⩽63 |
| BRCS k | If C==1 then PC ← PC+k+1 else PC ← PC+1 | -64⩽k⩽63 |

PC=0x200

```
        BRCC  loop
loop1:  ADD  R16, R17   If C=1   PC=0x201
        ……
loop:   AND R16, R17    If C=0   PC=Loop
```

PC=0x200

```
        BRCS  loop
Loop1:  ADD  R16, R17   If C=0   PC=0x201
        ……
Loop:   AND R16, R17    If C=1   PC=loop
```

# Instruction Sets and example - 6

| Syntax | Operation | Operands |
| --- | --- | --- |
| BREQ k | If Rd == Rr (Z == 1)<br>then PC ← PC + k + 1<br>else PC ← PC + 1 | -64⩽k⩽63 |

```
        cp r1,r0 ; Compare registers r1 and r0
        breq equal ; Branch if registers equal
        ...
equal:  nop ; Branch destination (do nothing)
```

# Instruction Sets and example - 7

| Syntax | Operation | Operands |
| --- | --- | --- |
| BRGE k | If Rd ≥ Rr (S == 0)<br>then PC ← PC + k + 1<br>else PC ← PC + 1 | -64⩽k⩽63 |

```
        cp r11,r12 ; Compare registers r11 and r12
        brge greateq ; Branch if r11 ≥ r12 (signed)
        ...
greateq: nop ; Branch destination (do nothing)
```

# Instruction Sets and example - 8

| Syntax | Operation | Operands | Program Counter | Stack |
|---|---|---|---|---|
| CALL k | PC ← k Devices with 16-bit PC, 128 KB program memory maximum | 0⩽k⩽64K | PC ← k | Stack ← PC+2<br>SP ← SP-2<br>2bytes, 16bits |
| CALL k | PC ← k Devices with 22-bit PC, 8 MB program memory maximum | 0⩽k⩽4M | PC ← k | Stack ← PC+2<br>SP ← SP-3<br>3bytes, 22bits |

```
mov r16,r0 ; Copy r0 to r16
call check ; Call subroutine
nop ; Continue (do nothing)
 ...
check:  Brank condition
```

# Instruction Sets and example- 9

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| CP Rd, Rr | Rd - Rr | 0⩽d⩽31 0⩽r⩽31 | PC ← PC+1 |
| CPI Rd, K | Rd - K | 0⩽d⩽31 0⩽K⩽255 | PC ← PC+1 |

R4 = 0x10 R17=0x21 N=1 PC=0x200

CP R4, R17 N=1 PC=0x201

CPI R17, 0x20 N=0 PC=0x201

# Instruction Sets and example- 10

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| IN Rd, A | Rd ← I/O(A) | 0⩽d⩽31 0⩽A⩽63 | PC ← PC+1 |
| OUT A, Rr | I/O(A) ← Rr | 0⩽r⩽31 0⩽A⩽63 | PC ← PC+1 |

PC=0x200

```
in R25, 0x18        Read port B   PC=0x201

out 0x18,r16        ;Write zeros to Port B
```

# Instruction Sets and example- 11

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| LD Rd, X | Rd ← DS(X) | 0⩽d<31 | PC ← PC+1 |
| LD Rd, X+ | Rd ← DS(X) X ←X+1 | 0⩽d<31 | PC ← PC+1 |
| LD Rd, -X | X ← X-1 Rd ← DS(X) | 0⩽d<31 | PC ← PC+1 |
| ST X, Rr | DS(X) ← Rr | 0⩽r<31 | PC ← PC+1 |
| ST X+, Rr | DS(X) ← Rr X ← X+1 | 0⩽r<31 | PC ← PC+1 |
| ST -X, Rr | X ← X-1 DS(X) ← Rr | 0⩽r<31 | PC ← PC+1 |

```
ldi r26,0x60 ; Set X low byte to 0x60
ld r0,X+ ; Load r0 with data space loc. 0x60(X post inc)
ld r1,X ; Load r1 with data space loc. 0x61
ldi r26,0x63 ; Set X low byte to 0x63
ld r2,X ; Load r2 with data space loc. 0x63
ld r3,–X ; Load r3 with data space loc. 0x62(X pre dec)
```

# Instruction Sets and example- 12

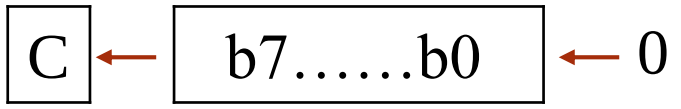
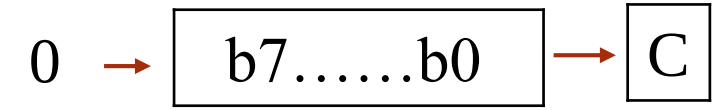
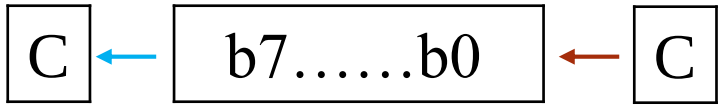
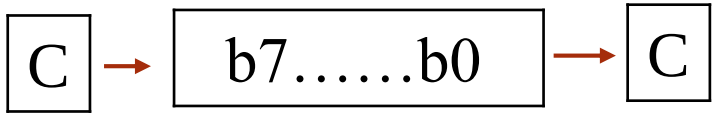
| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| LDI Rd, K | Rd ← K | 16⩽d⩽31 0⩽K⩽255 | PC ← PC+1 |
| LDS Rd, k | Rd ← DS(k) | 0⩽d⩽31 0⩽k⩽65535 | PC ← PC+2 |
| LDS Rd, k | Rd ← DS(k) | 16⩽d⩽31 0⩽k⩽127 | PC ← PC+1 |

ldi r30,0xF0 ; Load 0xF0 to r30

lds r2,0xFF00 ; Load r2 with the contents of data space location 0xFF00

lds r16,0x00 ; Load r16 with the contents of data space location 0x00

# Instruction Sets and example- 13

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| LSL Rd |  | 0⩽d⩽31 | PC ← PC+1 |
| LSR Rd |  | 0⩽d⩽31 | PC ← PC+1 |
| ROL Rd |  | 0⩽d⩽31 | PC ← PC+1 |
| ROR Rd |  | 0⩽d⩽31 | PC ← PC+1 |



lsl r0 ; Multiply r0 by 2

lsr r0 ; Divide r0 by 2

# Instruction Sets and example- 14

| Syntax | Operation | Operands | Program Counter |
|---|---|---|---|
| MOV Rd, Rr | Rd ← Rr | $0 \leqslant d \leqslant 31$<br>$0 \leqslant r \leqslant 31$ | PC ← PC+1 |
| MOVW Rd+1:Rd, Rr+1:Rr | R[d+1]:Rd ← R[r+1]:Rr | $d \in \{0,2,\ldots 30\}$<br>$r \in \{0,2,\ldots 30\}$ | PC ← PC+1 |

mov r16,r0 ; Copy r0 to r16

mov w r17:16,r1:r0 ; Copy r1:r0 to r17:r16

# Instruction Sets and example- 15

| Syntax | | Operation | Operands | Program Counter |
|---|---|---|---|---|
| MUL Rd, Rr | Unsigned | R1:R0 ← Rd × Rr | 0⩽d⩽31<br>0⩽r⩽31 | PC ← PC+1 |
| MULS Rd, Rr | Signed | R1:R0 ← Rd × Rr | 16⩽d⩽31<br>16⩽r⩽31 | PC ← PC+1 |
| MULSU Rd, Rr | Signed<br>Unsigned | R1:R0 ← Rd × Rr | 16⩽d⩽23<br>16⩽r⩽23 | PC ← PC+1 |

mul r5,r4 ; Multiply unsigned r5 and r4

muls r21,r20 ; Multiply signed r21 and r20

mulsu r23, r20 ; Multiply signed r23 and unsigned r20

# Instruction Sets and example- 16

| Syntax | Operation | Operands | Stack |
|---|---|---|---|
| RCALL k | PC ← PC+k+1 | -2K⩽k⩽2K | STACK ← PC+1<br>SP ← SP-2 (2byte) |
| RCALL k | PC ← PC+k+1 | -2K⩽k⩽2K | STACK ← PC+1<br>SP ← SP-3 (3byte) |

# Instruction Sets and example- 17

| Syntax | Operation | Operands | Stack |
|---|---|---|---|
| RET | PC(15:0)← STACK | none | SP ← SP+2 (2byte) |
| RET | PC(21:0)← STACK | none | SP ← SP+3 (3byte) |

# Instruction Sets and example- 18

| Syntax | Operation | Operands | Stack |
|---|---|---|---|
| RETI | PC(15:0) ← STACK | none | SP ← SP+2 (2byte) |
| RETI | PC(21:0) ← STACK | none | SP ← SP+3 (3byte) |

# Instruction Sets and example- 19

| Syntax | Operation | Operands |
|---|---|---|
| RJMP k | PC ← PC+k+1 | k-2K⩽k⩽2K |
| JMP k | PC ← k | 0⩽k⩽4M |

```
    rjmp ok ; Unconditional branch
    error: add r16,r17 ; Add r17 to r16
    inc r16 ; Increment r16
ok: nop ; Destination for rjmp (do nothing)
```

```
    jmp farplc ; Unconditional jump
    ...
farplc: nop ; Jump destination (do nothing)
```

# Instruction Sets and example- 20

| Syntax | | Operation | Operands |
|---|---|---|---|
| SBI A, b | Set Bit in I/O Register | I/O(A,b) ← 1 | 0⩽A⩽31<br>0⩽b⩽7 |
| SBR Rd, K | Set Bits in Register | Rd ← Rd∨K | 16⩽d⩽31<br>0⩽K⩽255 |

Skip if Bit in I/O Register Cleared**如果**I/O**寄存器中的位已清除，**则跳过

| | | | |
|---|---|---|---|
| SBIC A, b | Skip if Bit in I/O Register is Cleared | If I/O(A,b)==0 then PC ← PC+2(or 3)<br>else PC ← PC+1 | 0⩽A⩽31<br>0⩽b⩽7 |

Skip if Bit in I/O Register is Set如果设置了I/O**寄存器位，**则跳过

| | | | |
|---|---|---|---|
| SBIS A, b | Skip if Bit in I/O Register is Set | If I/O(A,b)==1 then PC ← PC+2(or 3)<br>else PC ← PC+1 | 0⩽A⩽31<br>0⩽b⩽7 |

Skip if Bit in Register Cleared跳过如果位在寄存器清除

| | | | |
|---|---|---|---|
| SBRC Rr, b | Skip if Bit in Register is Cleared | If Rr(b)==0 then PC ← PC+2(or 3)<br>else PC ← PC+1 | 0⩽r⩽31<br>0⩽b⩽7 |

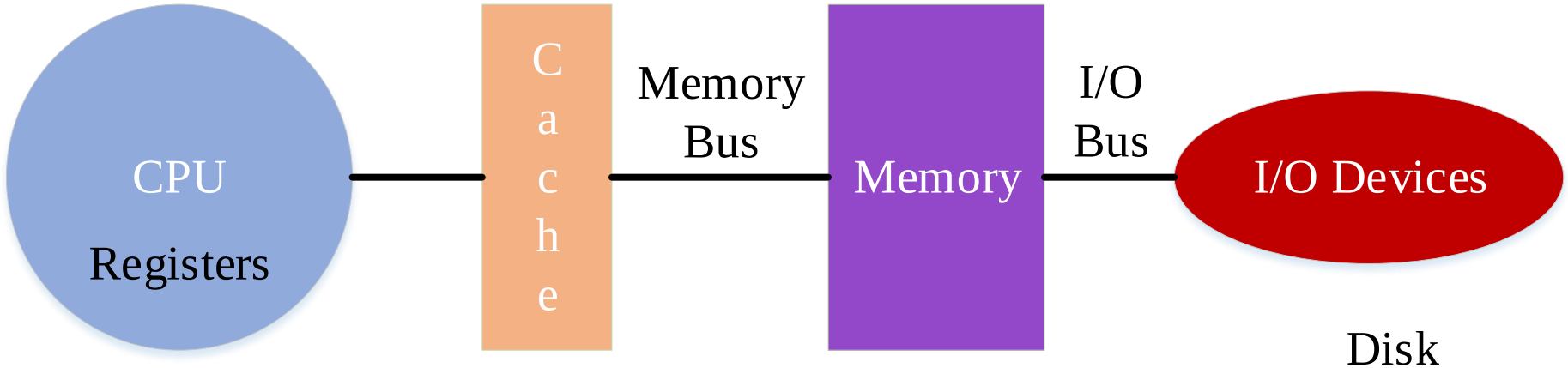
Skip if Bit in Register is Set如果寄存器位设置为设置，则跳过

| | | | |
|---|---|---|---|
| SBRS Rr, b | Skip if Bit in Register is Set | | |

# Instruction Set Architecture

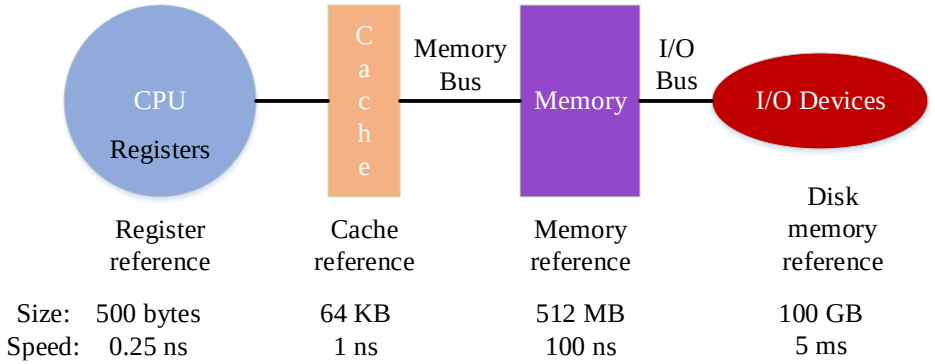
- How about CISC?
- CISC has become a catch - all term meaning anything that's not a “load - store” architecture (as in RISC) , but a "load - operate" one that allows arithmetic instructions to perform memory accesses as well.

# Memory Hierarchy - I



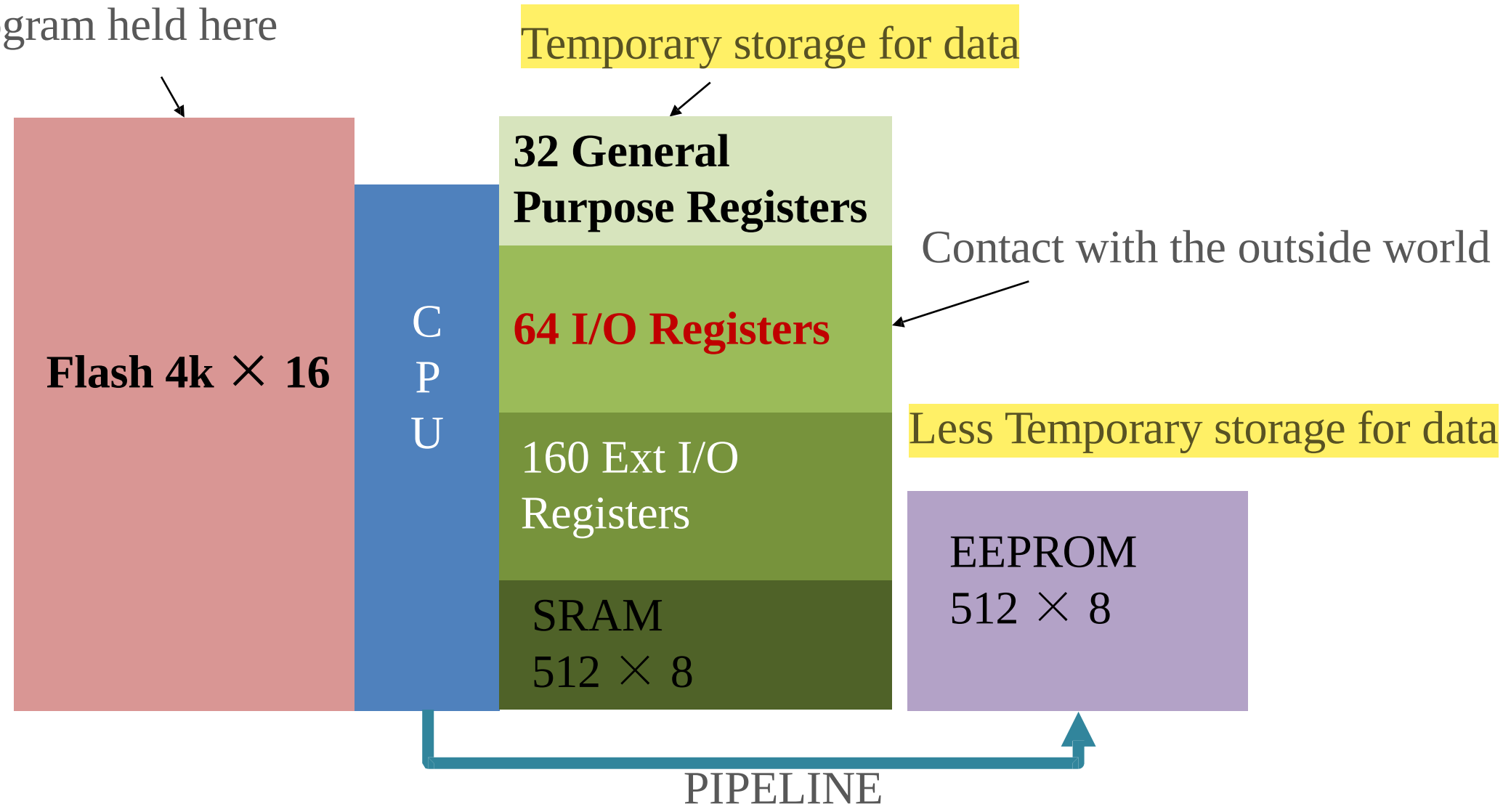
# Memory Hierarchy - II

- The CPU registers are by far the fastest but are also expensive in materials and implementation. Therefore they are very small.

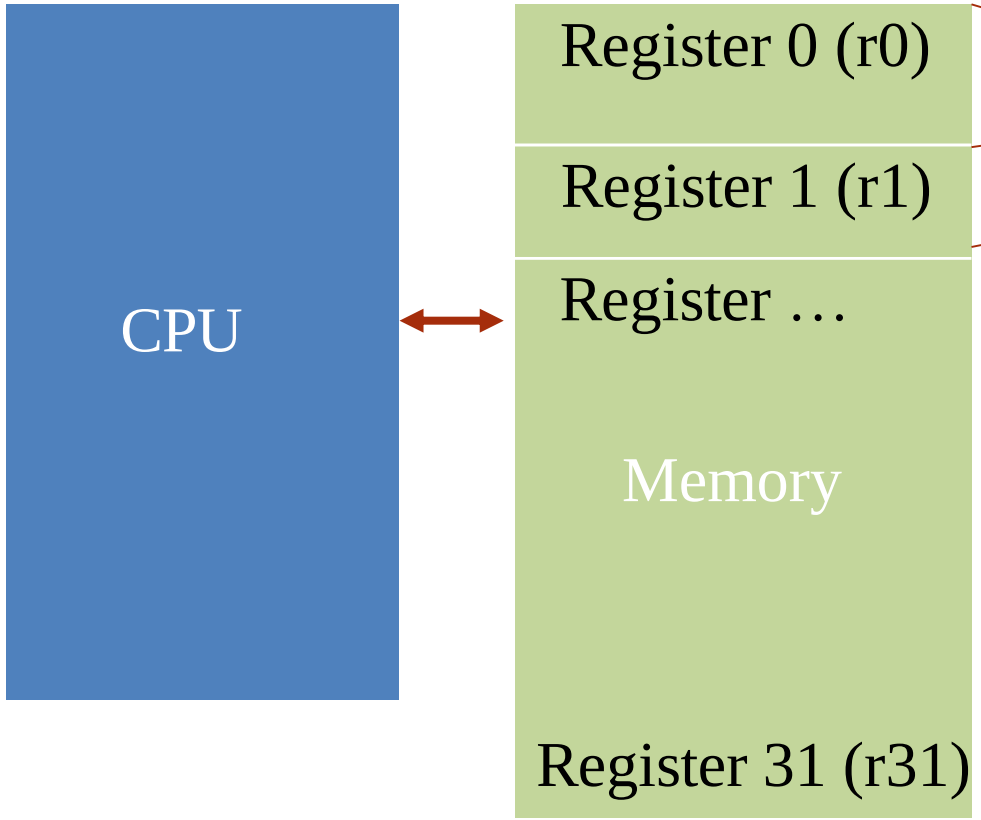


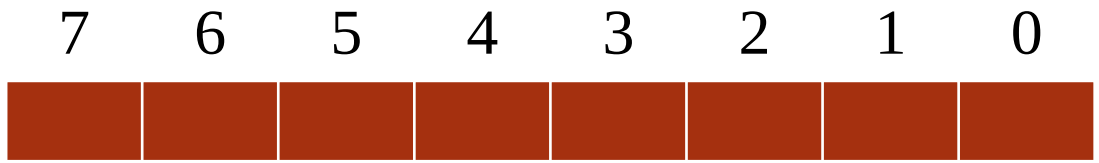

- The cache is usually on the die of the CPU, making it very fast but also expensive to implement since it uses SRAM (*Static* RAM –more MOS gates than DRAM – *Dynamic* RAM). The further away from the CPU the memory gates than DRAM. The further away from the CPU the memory block is (the 'lower' it is) the slower it is. This is due to the use of progressively cheaper technologies. So the size of the memory increases dramatically in devices lower down the hierarchy.

# ATMega Memory - map



# General Purpose Registers




- Short term (or working) memory of the CPU
- Slices of memory called registers
- Very fast access from the CPU
- Instructions operate directly on this memory
- Labelled as r0 – r31

# General Purpose Registers

- There are 32 standard registers (R0 – R31)
- 8 bits per register, of which each can be 0 or a 1
- A register can store 1 of:
  - Numbers
    - 0 – 255 [1111 1111]  (8 bit number)
    - - 128 – 127 [0111 1111] (signed 8 bit number)
  - ASCII encoded character (8 bit code)

# Register Commands

- Useful to manipulate these registers
- To load a value in to your register
  - ***LDI** r16*, 150
  - **LoaD I**mmediately 150 into r16

- Copy a register value
  - ***MOV** r17, r16*

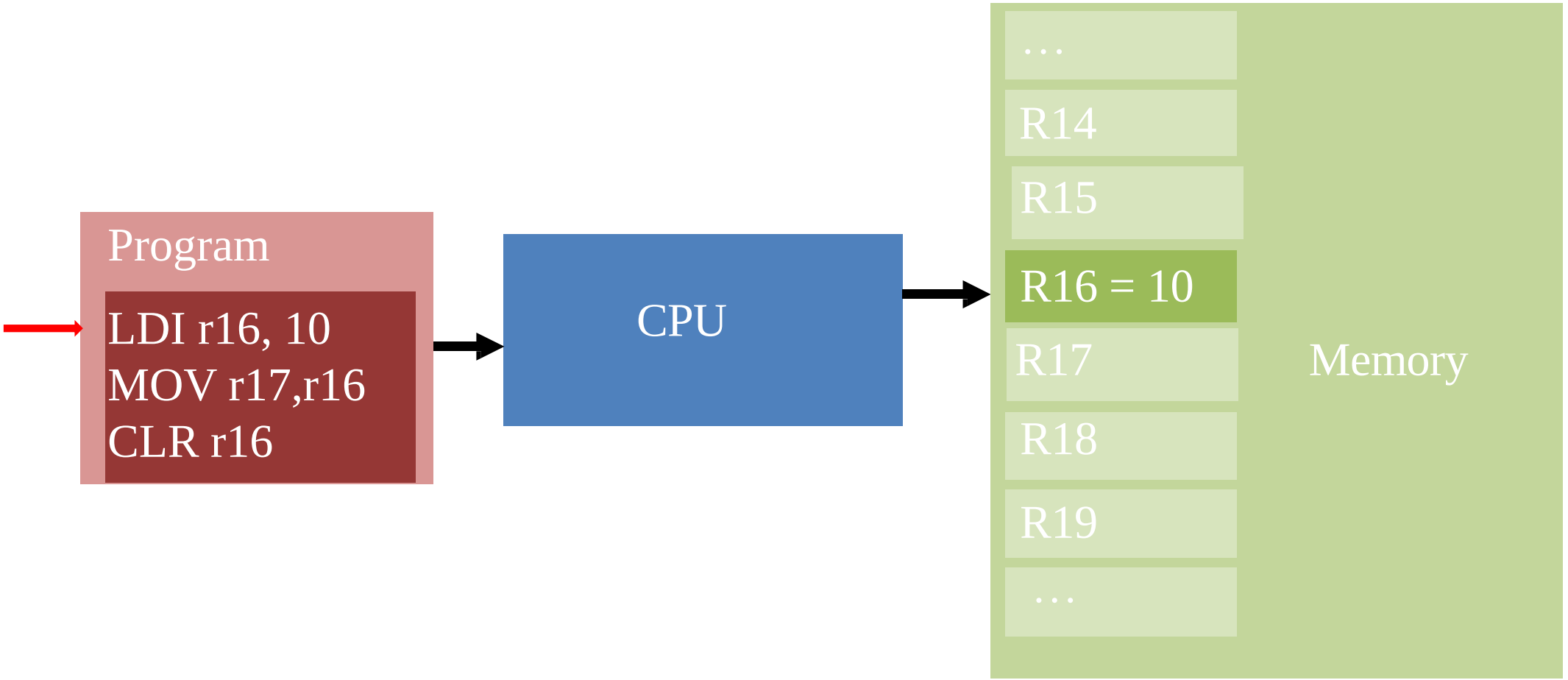
  **Copy**(!) value from r16 to r17
- Clear a register
  - *CLR r16*
  - **CL**ea**R** the value from r16




- For register commands the **DESTINATION** register is first.

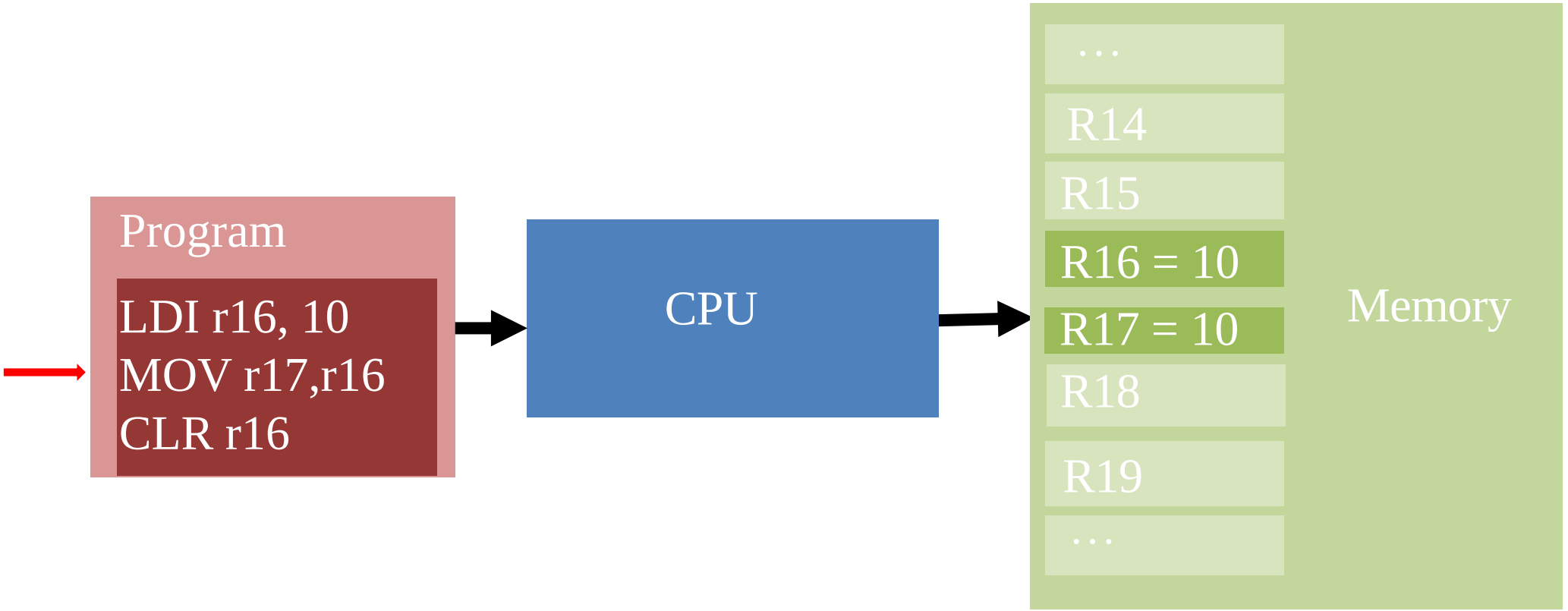
# LDI

- Set a value to a register (10 to r16)
- Only works for r16 to r31!



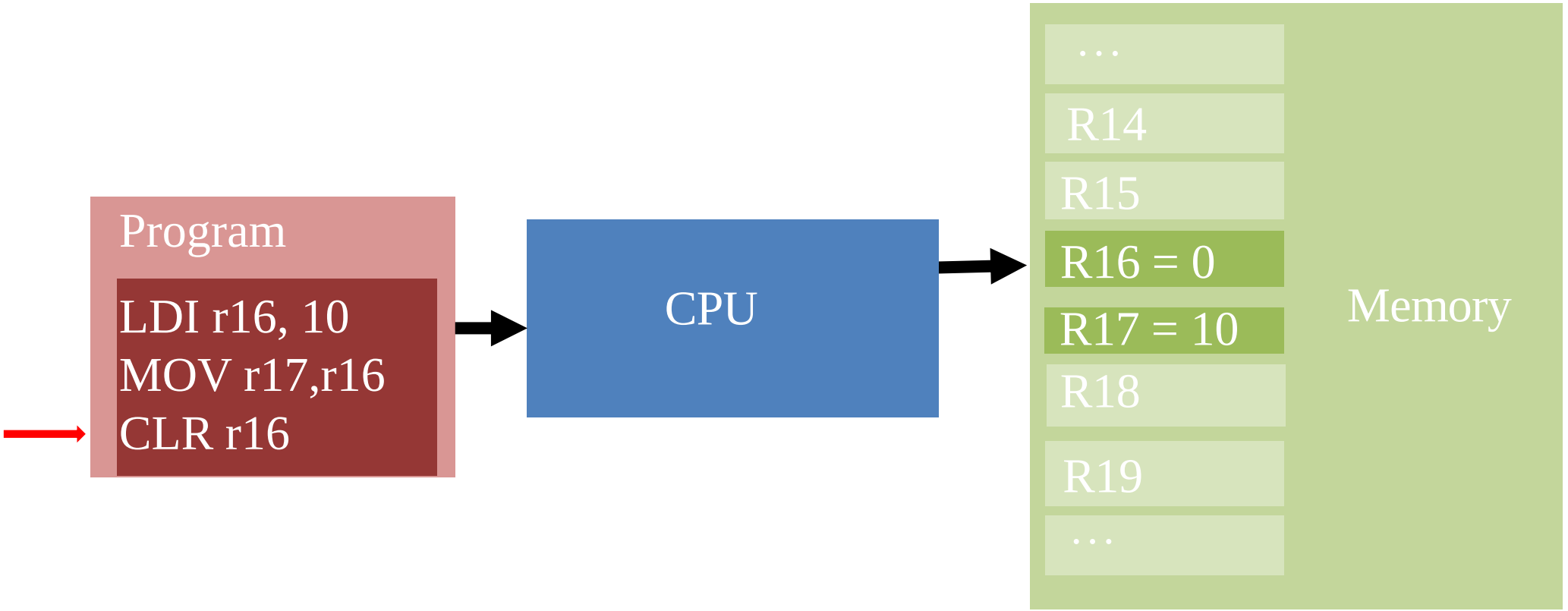
# MOV

- Copy this to register 17
- We have two registers with values in them



# CLR

- Clear register r16



# Limitations

- Not all registers are compatible with all commands
- Register R16 - R31 are compatible with **all** commands
- Register R0 - R15 are not…
  - LDI, ANDI, CBR, CPI, SBCI, SBR, SER, SUBI, for instance won't work

# Next Lecture

- Input and output!